

# 명암 조절

- 화면의 밝기(Luminance)를 밝거나 어둡게 조정
- 감마 조절(gamma correction)
  - 화면의 밝기를 인간의 시각 특성에 맞게 비선형적으로 조절하는 과정
  - 사람은 어두운 곳에서의 변화에는 민감하게 반응하지만, 아주 밝은 곳에서의 변화에는 상대적으로 둔감
  - 아래의 수식에서  $\gamma$  (감마, gamma)값으로 화면의 밝기를 조절

$$f'(j,i) = (L-1) \times \dot{f}(j,i)^\gamma$$

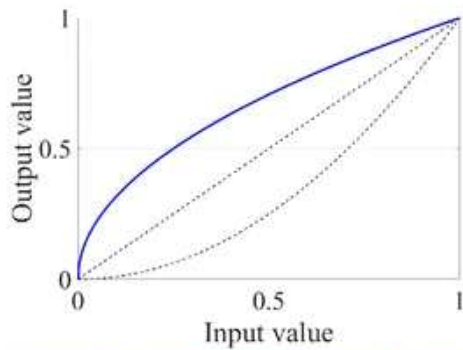
$L$  = 보통 256

$\dot{f}$  : 0~255 사이 밝기를 0~1사이로 정규화

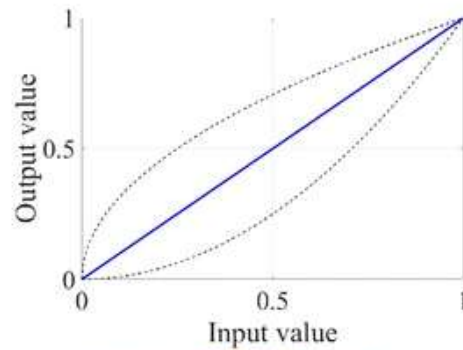


# 명암 조절

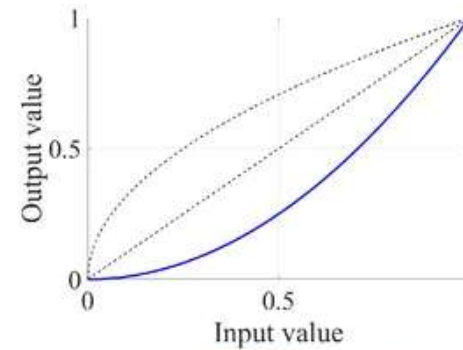
- 화면의 밝기(Luminance)를 밝거나 어둡게 조정
- 감마 조절(gamma correction)



$\gamma = 0.5$



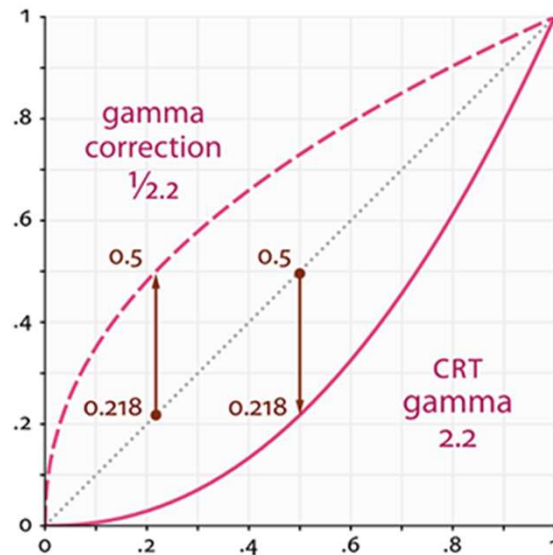
$\gamma = 1$



$\gamma = 2$

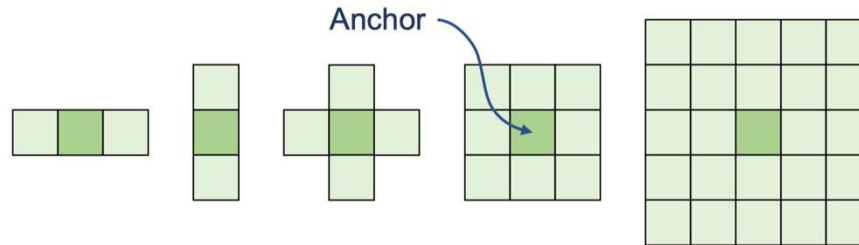
# 명암 조절

- 감마 조절(gamma correction)
  - 일반적인 모니터는 화면을 어둡게 출력하는 성질( $\gamma \approx 2.2$ )이 있음
  - 이를 상쇄하기 위해 영상을 만들 때 미리 화면을 그만큼 밝게( $\gamma \approx 1/2.2$ ) 만들어서 저장
  - 두 과정을 거치면  $1/2.2 \times 2.2 \approx 1$  이 되어, 우리 눈에 원래의 자연스러운 밝기로 보이게 됨



# 모폴로지(Morphology)

- 이미지 내에서 객체의 모양이나 구조를 분석하고 조작하는 기법
- 구조 요소(structuring element)를 이용하여 영역의 모양을 조작



구조요소의 예

- 기본 연산
  - 팽창(dilation) : 작은 홈을 메우거나 끊어진 영역을 연결하는 효과. 영역을 키움
  - 침식(erosion) : 경계에 솟은 돌출 부분을 깎는 효과. 영역을 작게 만듦
  - 열림(opening) : 침식한 결과에 팽창 적용. 원래 영역 크기 유지
  - 닫힘(closing) : 팽창한 결과에 침식을 적용. 원래 영역 크기 유지

# 모폴로지(Morphology) 예시

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 | 1 | 1 | 1 | 0 |



구조 요소

입력 영상

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 | 1 | 1 | 1 | 0 |

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |

팽창

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 | 1 | 1 | 1 | 0 |

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |

침식



잘라낸 영상

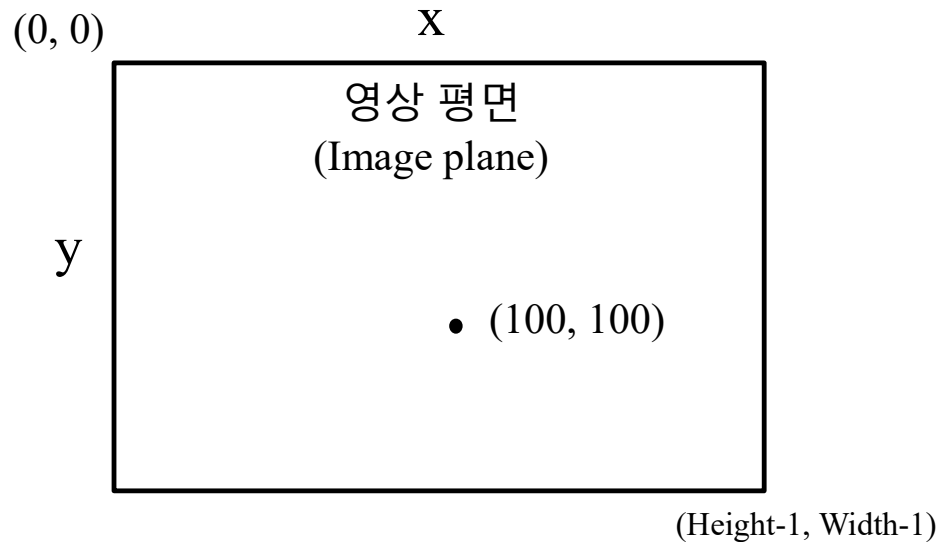
팽창

침식

단힘

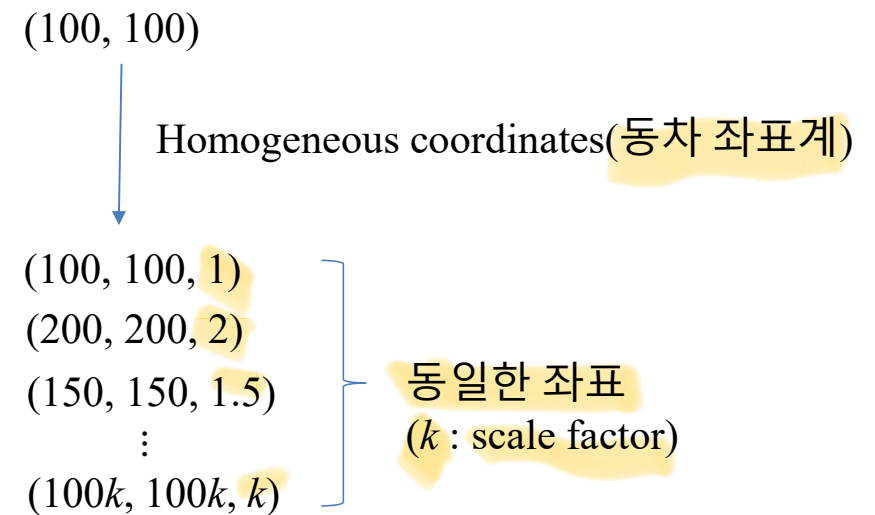
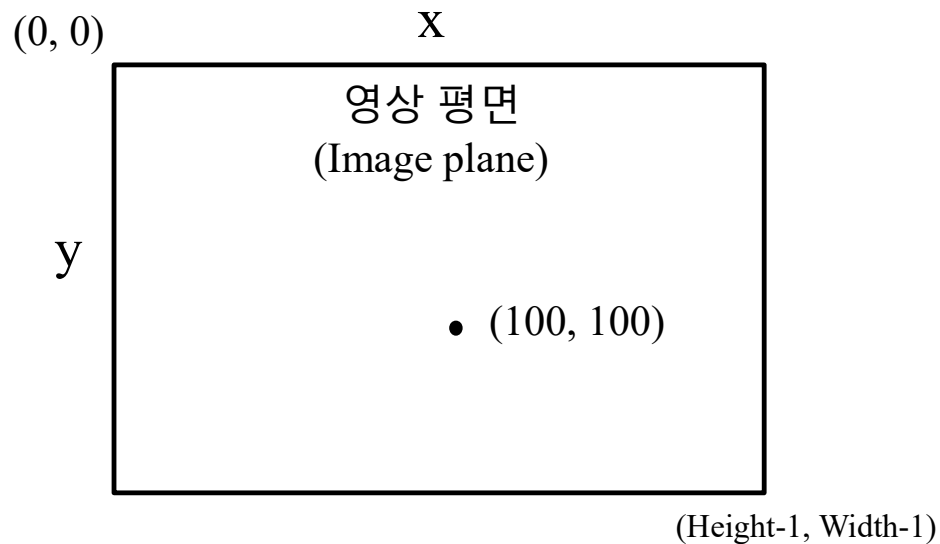
# 기하 변환

- Homogeneous coordinates(동차 좌표계)
  - 2차원 좌표에 1을 추가해 3차원 벡터로 표현
  - 3개 요소에 같은 값을 곱하면 같은 좌표. 예)  $(-2, 4, 1)$ 과  $(-4, 8, 2)$ 는  $(-2, 4)$ 에 해당



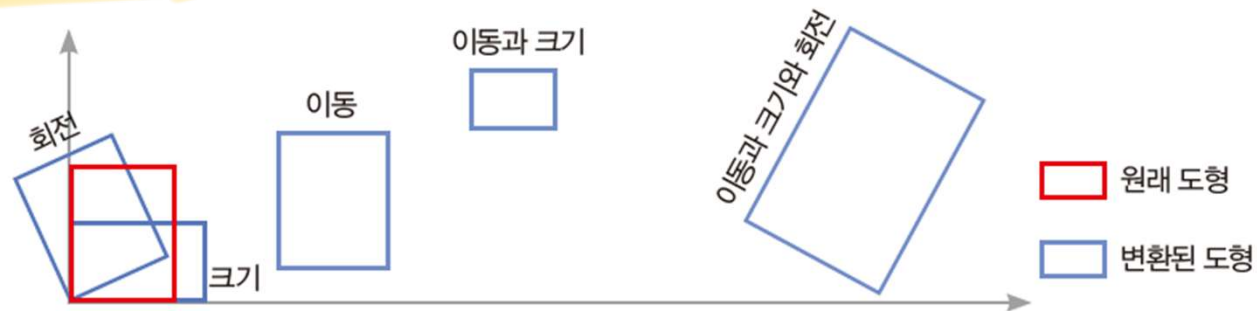
# 기하 변환

- Homogeneous coordinates(동차 좌표계)
  - 2차원 좌표에 1을 추가해 3차원 벡터로 표현
  - 3개 요소에 같은 값을 곱하면 같은 좌표. 예)  $(-2, 4, 1)$ 과  $(-4, 8, 2)$ 는  $(-2, 4)$ 에 해당



# 기하 변환

- 기하 변환의 종류

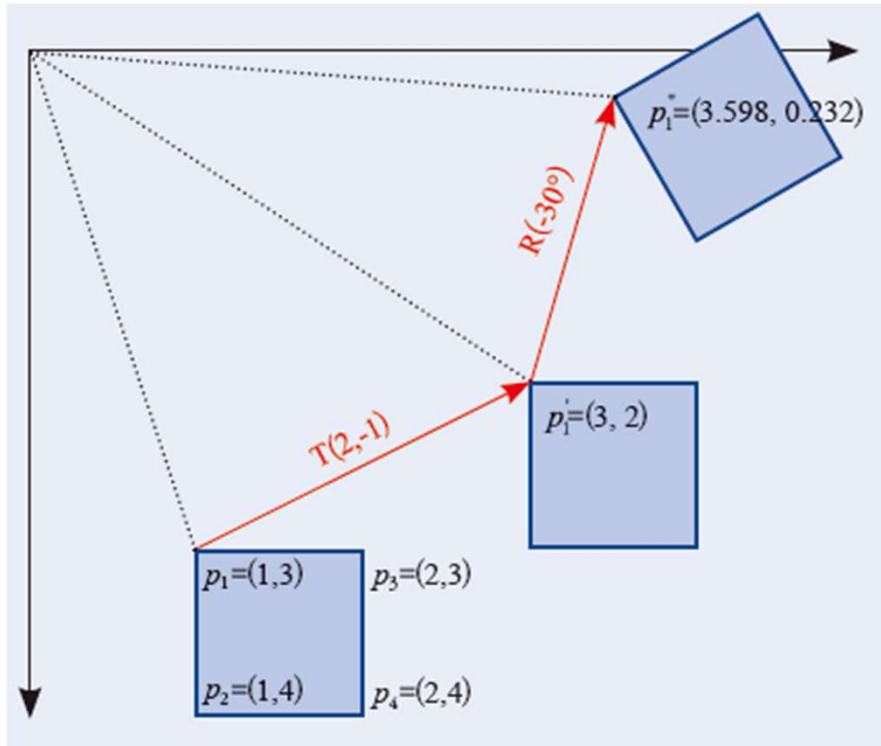


|    |                                                                                                                          |                                          |
|----|--------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| 이동 | $T(t_x, t_y) = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$                                    | $x$ 방향으로 $t_x$ , $y$ 방향으로 $t_y$ 만큼 이동    |
| 회전 | $R(\theta) = \begin{pmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$ | 원점을 중심으로 반시계 방향으로 $\theta$ 만큼 회전         |
| 크기 | $S(s_x, s_y) = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$                                    | $x$ 방향으로 $s_x$ , $y$ 방향으로 $s_y$ 만큼 크기 조정 |



# 기하 변환

- 기하 변환의 예
  - 정사각형을  $x$  방향으로 2,  $y$  방향으로 -1만큼 이동한 다음 반시계 방향으로 30도 회전



$$T(2, -1) = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix}, \quad R(30^\circ) = \begin{pmatrix} 0.8660 & 0.5000 & 0 \\ -0.5000 & 0.8660 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

이동 적용

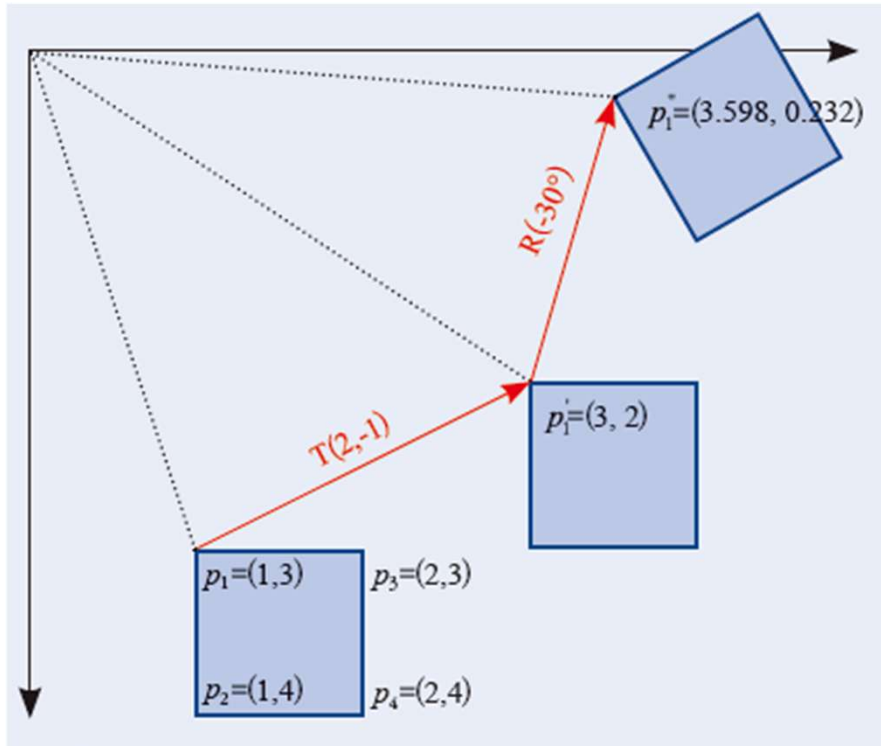
$$\bar{p}_1'^T = T(2, -1) \bar{p}_1^T = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 2 \\ 1 \end{pmatrix}$$

회전 적용

$$\bar{p}_1''^T = R(30^\circ) \bar{p}_1'^T = \begin{pmatrix} 0.8660 & 0.5000 & 0 \\ -0.5000 & 0.8660 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 3 \\ 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 3.598 \\ 0.232 \\ 1 \end{pmatrix}$$

# 기하 변환

- 기하 변환의 예
  - 정사각형을  $x$  방향으로 2,  $y$  방향으로 -1만큼 이동한 다음 반시계 방향으로 30도 회전



- 복합 변환을 위한 행렬을 미리 곱해 놓으면 모든 점에 대해 한번의 행렬 곱셈으로 기하 변환 가능

동차 좌표계의 장점 (Homogeneous)

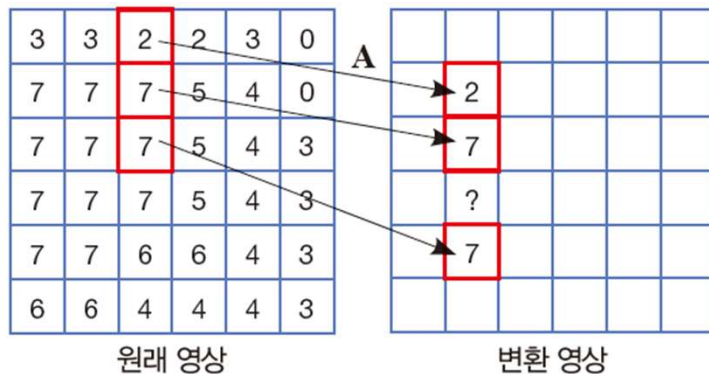
$$A = R(30^\circ)T(2, -1) = \begin{pmatrix} 0.8660 & 0.5000 & 0 \\ -0.5000 & 0.8660 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0.8660 & 0.5000 & 1.232 \\ -0.5000 & 0.8660 & -1.866 \\ 0 & 0 & 1 \end{pmatrix}$$

$$A\bar{p}_1^T = \begin{pmatrix} 0.8660 & 0.5000 & 1.232 \\ -0.5000 & 0.8660 & -1.866 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 3.598 \\ 0.232 \\ 1 \end{pmatrix}$$

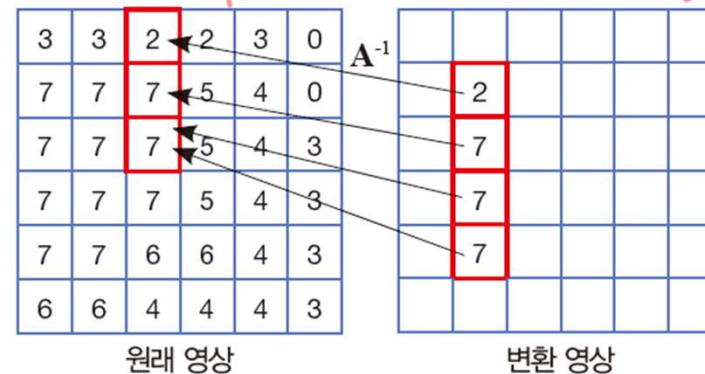
# 기하 변환

## Forward mapping and Backward mapping

- Forward mapping 때는 변환 영상에서 값을 받지 못하는 픽셀이 생기는 문제 발생
- Backward mapping 방식으로 문제 해결 가능



Forward mapping (전방 변환)



Backward mapping (후방 변환)

하다보니  
값을 받지 못하는  
픽셀이 생김..

# OpenCV 개요

- 컴퓨터 비전과 이미지 처리를 목적으로 하는 오픈소스 라이브러리
- 2000년 인텔리서치에서 공개
- 현재는 OpenCV Foundation에서 비영리 운영
- 함수와 클래스는 C/C++로 개발
- 인터페이스 언어는 C, C++, 자바, 자바스크립트, 파이썬 지원
- 교육 목적, 상용 목적 모두 무료 사용 가능(Apache 2.0)
- 가장 최신 버전은 OpenCV 4.x
- 공식 홈페이지 : <https://opencv.org>
- 매뉴얼 사이트 : <https://docs.opencv.org/>
- 공식 깃허브 : <https://github.com/opencv/opencv>

# 실습 환경

- 구글 코랩(Google Colab)
  - 코랩에는 opencv-python 라이브러리가 이미 설치되어 있음
  - cv2.imshow() 함수를 사용하면 에러가 발생하므로 코랩 전용 출력 함수를 사용해야 함
  - 웹상의 이미지를 내려받거나 영상 파일을 업로드해서 사용 가능

```
# OpenCV의 version 확인
!pip show opencv-python
```

# 영상처리 실습

- OpenCV 영상 출력

```
import urllib.request
from google.colab.patches import cv2_imshow

# URL 설정 (OpenCV 공식 깃허브 저장소 내 샘플 데이터)
url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/fruits.jpg'
save_name = 'fruits.jpg'

# 파일 다운로드
urllib.request.urlretrieve(url, save_name)

# 확인 출력
img = cv2.imread(save_name)
cv2_imshow(img)
```

# 영상처리 실습

- OpenCV에서 영상은 `numpy.ndarray` 클래스 형의 객체임
- 그러므로 OpenCV가 다루는 영상은 numpy가 제공하는 다양한 기능(함수)을 사용할 수 있음
  - Matplotlib으로 시각화하거나 Scikit-learn, PyTorch, TensorFlow 같은 딥러닝/머신러닝 라이브러리에 바로 적용 가능
  - NumPy의 슬라이싱 기능 등을 사용하면 이미지의 특정 영역(ROI, Region of Interest)을 잘라내거나 수정하는 작업 용이
- Numpy는 내부적으로 C와 포트란으로 구현되어 있어 파이썬의 기본 리스트보다 매우 빠름

```
pip show numpy # numpy가 설치되어 있는지 확인
```

```
import numpy as np #numpy import
```

```
# 영상의 배열 정보 확인
```

```
img.shape # (480, 512, 3)
```

```
img.ndim # 3
```

```
img.size # 737280
```

```
img.dtype # dtype('uint8')
```

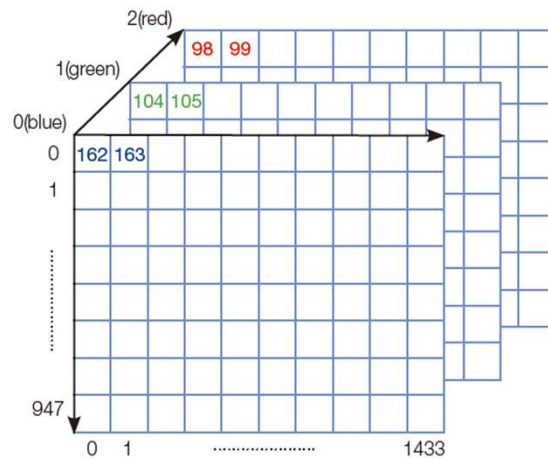
(3/2 x 480 영상)

영상처리 할 때는

512 ↔ 480 자리 뒤바꿈

# 영상처리 실습

- OpenCV에서의 영상 데이터 구조



```
print(img[0,0,0], img[0,0,1], img[0,0,2])    # (0, 0) 픽셀값
```

```
print(img[0,1,0], img[0,1,1], img[0,1,2])    # (0, 1) 픽셀값
```

RGB 순서가 아니라, 뒤바뀐 순서로 저장

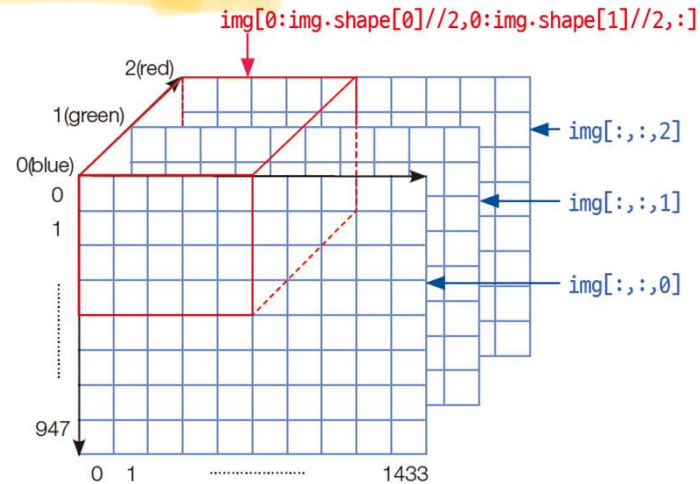
※ OpenCV에서는 BGR의 순서로 픽셀 데이터를 저장

```
In [3]: print(img[0,0,0], img[0,0,1], img[0,0,2])
        162 104 98
In [4]: print(img[0,1,0], img[0,1,1], img[0,1,2])
        163 105 99
```



# 영상처리 실습

- OpenCV에서의 슬라이싱(Slicing)



```
B_channel = img[ :, :, 0] # blue channel
G_channel = img[ :, :, 1] # green channel
R_channel = img[ :, :, 2] # red channel
```

```
cv2_imshow(R_channel)
cv2_imshow(G_channel)
cv2_imshow(B_channel)
```

```
roi = img[100:300, 200:400]
cv2_imshow(roi)
```

```
downsampled = img[:, ::2, ::2]
cv2_imshow(downsampled)
```

# 내용 정리

- 2D 영상의 특성 및 영상 처리 기술
- 영상의 품질 측정 지표
- OpenCV 소개 및 영상 데이터 구조

## 다음 주 강의 내용

- 영상의 변환(Transform) 및 필터링
- 에지 및 코너 탐지
- 영상의 품질 측정 지표